NAME:-Pradeep Lahu Rathod

ROLL NO:-49

EXPERIMENT NO:-8

BELL MAN FORD :

CODE:-

```
// Bellman Ford Algorithm in C


#include <stdio.h>

#include <stdlib.h>


#define INFINITY 99999


//struct for the edges of the graph

struct Edge {

  int u;  //start vertex of the edge

  int v;  //end vertex of the edge

  int w;  //weight of the edge (u,v)

};


//Graph - it consists of edges

struct Graph {

  int V;       //total number of vertices in the graph

  int E;       //total number of edges in the graph

  struct Edge *edge;  //array of edges

};
```

```
void bellmanford(struct Graph *g, int source);

void display(int arr[], int size);


int main(void) {

  //create graph

  struct Graph *g = (struct Graph *)malloc(sizeof(struct Graph));

  g->V = 4;  //total vertices

  g->E = 5;  //total edges


  //array of edges for graph

  g->edge = (struct Edge *)malloc(g->E * sizeof(struct Edge));


  //------- adding the edges of the graph

  /*

                edge(u, v)

                where   u = start vertex of the edge (u,v)

                                v = end vertex of the edge (u,v)


                w is the weight of the edge (u,v)

          */


  //edge 0 --> 1

  g->edge[0].u = 0;

  g->edge[0].v = 1;
```

```
g->edge[0].w = 5;

//edge 0 --> 2
g->edge[1].u = 0;
g->edge[1].v = 2;
g->edge[1].w = 4;

//edge 1 --> 3
g->edge[2].u = 1;
g->edge[2].v = 3;
g->edge[2].w = 3;

//edge 2 --> 1
g->edge[3].u = 2;
g->edge[3].v = 1;
g->edge[3].w = 6;

//edge 3 --> 2
g->edge[4].u = 3;
g->edge[4].v = 2;
g->edge[4].w = 2;

bellmanford(g, 0);  //0 is the source vertex

return 0;
```

```
}

void bellmanford(struct Graph *g, int source) {
 //variables
 int i, j, u, v, w;

 //total vertex in the graph g
 int tV = g->V;

 //total edge in the graph g
 int tE = g->E;

 //distance array
 //size equal to the number of vertices of the graph g
 int d[tV];

 //predecessor array
 //size equal to the number of vertices of the graph g
 int p[tV];

 //step 1: fill the distance array and predecessor array
 for (i = 0; i < tV; i++) {
  d[i] = INFINITY;
  p[i] = 0;
 }
```

```
//mark the source vertex
d[source] = 0;

//step 2: relax edges |V| - 1 times
for (i = 1; i <= tV - 1; i++) {
  for (j = 0; j < tE; j++) {
    //get the edge data
    u = g->edge[j].u;
    v = g->edge[j].v;
    w = g->edge[j].w;

    if (d[u] != INFINITY && d[v] > d[u] + w) {
      d[v] = d[u] + w;
      p[v] = u;
    }
  }
}

//step 3: detect negative cycle
//if value changes then we have a negative cycle in the graph
//and we cannot find the shortest distances
for (i = 0; i < tE; i++) {
  u = g->edge[i].u;
  v = g->edge[i].v;
```

```
    w = g->edge[i].w;
    if (d[u] != INFINITY && d[v] > d[u] + w) {
      printf("Negative weight cycle detected!\n");
      return;
    }
  }

  //No negative weight cycle found!
  //print the distance and predecessor array
  printf("Distance array: ");
  display(d, tV);
  printf("Predecessor array: ");
  display(p, tV);
}

void display(int arr[], int size) {
  int i;
  for (i = 0; i < size; i++) {
    printf("%d ", arr[i]);
  }
  printf("\n");
}
```

OUTPUT:-

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE COMMENTS

```
Warning: Debuggee TargetArchitecture not detected, assuming x86_64.
=cmd-param-changed,param="pagination",value="off"
=cmd-param-changed,param="args",value="2>CON 1>CON <CON"
[New Thread 17788.0x4bb0]
[New Thread 17788.0x1888]
Loaded 'C:\WINDOWS\SysWOW64\kernel32.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\SysWOW64\KernelBase.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\SysWOW64\apphelp.dll'. Symbols loaded.
Loaded 'C:\WINDOWS\SysWOW64\msvcrt.dll'. Symbols loaded.
Distance array: 0 5 4 8
Predecessor array: 0 0 0 1
The program 'C:\TurboC++\Disk\TurboC3\BIN\KetanAOA\insertionsort.exe' has exited with code 0 (0x00000000).
```